

深度学习问题与证据实验

先用纸笔建立检查基准

本实验比较两条已经完整定义的预测规则，不训练神经网络。目标是把一份含糊的“模型不错”改成可复核的数值与范围。只需加减乘除；Python 的逐行写法会在002单元讲解。没有 Python 经验的读者可以先完成前三节纸笔任务，再把整本 Notebook 当作核验器运行。

实验只使用本地原创合成数据与 Python 标准库，CPU 即可，不需要网络、GPU、付费API或账号。建议先读 lecture.pdf 的前五节，暂时不要打开测试标签文件。若已经看过标签，照样可以练习，但应将结果标成“公开答案流程复现”。

一 准备一张任务卡

在纸上写下六项内容：一次样本是一次校准记录；输入是旋钮位置；目标是屏幕读数；预测发生在屏幕结果出现之前；评分使用平均绝对误差；测试资料不参与修改本轮规则。再写明数据是合成的，结果只用于教学。

检查一个容易犯的错误：输入表中若有“屏幕结果复制列”，不能因为它能读入程序就使用。你应对照预测时点判断它何时可用。文件里有某列与预测时可以读取某列，是两回事。

二 认识文件

文件	用途
lecture.pdf 与 lecture.md	概念、计算与图解
lab.pdf 与 lab.md	本实验的操作与验收
answers.pdf 与 answers.md	讲义练习完整推理
data/train.csv	五条可用于选择偏移量的记录
data/test_inputs.csv	四个待预测输入，不含标签
data/test_labels.csv	按编号对应的公开教学测试答案
data/stress_labels.csv	关系改变后的合成压力测试答案
experiment.py	完整可读实验程序及输入检查
experiment.ipynb	按步骤解释并核对结果的实验本

CSV是文本表格：第一行写列名，每个逗号分隔一个字段。编号T01不是数值，它是把一行预测与一行标签对应起来的名字。不要依靠肉眼认为两份文件行序一定相同；程序按编号检查，再按测试输入顺序输出。

运行脚本会在本讲目录创建 outputs 文件夹，保存选出的偏移量、逐条预测、汇总指标与版本记录。重复运行会更新同名输出，原始 data 文件不变。若要保留自己的改动，复制完整单元文件夹后再实验。

三 冻结两种规则并写下预期

先打开训练表。规则A固定系数2，只比较偏移量0、1、2。用下面步骤逐一算：对每行把输入乘2再加偏移量；减去标签并取绝对值；五条相加再除以5。预期训练 MAE 依次是1.2、0.2、0.8，因而选择1。若出现同分，取偏移量更小者。

规则B保存训练表，输入精确命中时返回已有标签，否则返回0。它的训练 MAE 应为0。请不要临时把“否则返回0”换成更有利的做法。那是另一条可以研究的规则，需要单独记录。

在揭示测试标签之前写下两条规则对四个测试输入的预测。A应输出4、6、8、10，B应输出0、0、0、0。将这张预测表保留。预测不是先看标签再倒填的答案。

现在打开测试标签。A的总绝对误差0，平均0；B的总误差28，平均7。再打开压力测试标签，保持A不变，核对其总误差12、平均3。压力测试是人为改变关系的反例，不代表实测了另一台设备。

四 运行程序

尚未安装环境时，可以先跳到002，按官方安装入口建立独立环境，再回来运行。已有 Python 3.12 的读者不需额外科学计算库。终端是接收命令的窗口；先切换到本讲所在文件夹，再执行：

```
python experiment.py
```

不知道当前目录是否正确时，先在文件管理器确认这里能看到 experiment.py 和 data 文件夹。命令中的文件名不要多加 .txt。若出现“找不到文件”，先核对位置，不要删除或搬动系统文件。002会详细解释终端、解释器和路径。

程序先读取训练记录，选择并保存偏移量与两种规则的预测，然后才读取测试标签进行核对。这种代码顺序演示信息隔离，但不构成安全封存：代码作者和公开仓库读者仍可查看答案。我们不会把同一人同时持有的公开文件称为真正的外部盲测。

正常结束后，终端打印简洁的JSON结果，其中应包含 selected_bias 为1、A_train_mae 为0.2、B_train_mae 为0、A_test_mae 为0、B_test_mae 为7、A_stress_mae 为3。JSON是一种给字段命名的文本记录，本讲只需按名称找到对应数值，不要求自己编写。

输出文件 frozen_predictions.json 记录看答案之前的参数与预测；predictions.csv 记录逐行标签、预测、误差；metrics.json 记录汇总；environment.json 只保存Python版本与平台名称，不保存用户名或私人路径。

五 运行 Notebook 并清空状态

在安装了 JupyterLab 的环境中输入 `jupyter lab`，在打开的文件浏览区进入本单元，打开 `experiment.ipynb`。从上到下运行说明之后的每个代码格。第一格只导入程序与标准库；后续格分别计算训练选择、冻结预测、揭示标签、执行故障检查。正文会告诉你应看到什么。

第一次成功后，使用 Restart Kernel and Run All Cells 或含义相同的“重启内核并运行全部”。重启会丢弃内存中的旧变量；全部重跑检验文件本身是否提供了足够步骤。不要仅凭页面保留的旧输出判定本次已经执行。

课程制作时使用全新Python进程中的真实 IPython InProcessKernel，顺序执行并保存全部代码单元输出。制作环境限制了套接字，因此未检验浏览器Jupyter界面、外进程内核传输或每个操作系统的 Anaconda安装。这一事实也记录在 `verification.json`，不能被“通过执行”几个字省略。

六 故意制造问题

先运行原样测试，确认不是环境本身有问题。然后在副本里做三种改动，每次只改一处，并写下你预计会发生什么。

第一种，把真实值列表清空。平均误差没有定义，程序应抛出 `ValueError`，而不是打印0。第二种，从标签列表中删掉一个数，保留预测长度不变。程序应拒绝长度不匹配，避免漏算。第三种，把输入或标签写成无法解释的值，例如 `nan`。程序应拒绝非有限数值；`nan`是浮点数中的特殊标记，不是一个正常测量结果。

配套 `run_self_tests` 函数还检查查表未命中、负向偏差转绝对值、重复训练输入，以及预测编号和标签编号不一致。捕捉到这些预定错误才算测试通过。“代码没有崩溃”不等于算法正确；“故障测试成功识别错误”也不能替代对公式和任务的检查。

七 提交什么算完成

保留一张手算表、两条冻结规则、完整逐条预测、六个汇总数和一段限制说明。限制至少包括：数据合成、测试仅四点、标签公开、没有真实装置实测、规则B的未命中选择很特殊。你还应能指出压力测试改变了关系，而不是修改了A的参数。

合格报告应让另一位读者不用猜测就能重算出0.2、0、0、7和3。若结果不同，先查输入顺序、偏移量、绝对值和除数，再查环境。不要为了“对上答案”直接把程序输出改成预期数字。

练习详解在 `answers.pdf`。先自己写完，再用详解检查推理；只复制最终数值无法检验是否理解了信息边界。